Cloud Pipeline Authentication with OIDC

Day 03 | AWS / AZURE / GCP / GITHUB ACTIONS

Use this guide to investigate a specific failure, apply a cause-specific change and verify recovery. Start with evidence from the affected workload. The examples require adaptation to your environment.

How the failure happens

OIDC separates identity proof from cloud authorization. A CI job presents a signed identity token; the cloud evaluates issuer, audience and subject before issuing temporary credentials. Role permissions then decide which resources the job can change.

The Scenario A deployment fails because its long-lived cloud credential expired.

The Alternative Federate the CI identity and exchange its OIDC token for short-lived cloud credentials.

Still Required Configure trust and authorization. OIDC does not grant permissions by itself.

Before you start

A controlled GitHub repository and deployment environment; cloud administrator setup for federation; least-privilege deployment roles; reviewed workflow changes.

Diagnostic checks

```
# Inspect login-step errors
# Check token issuer, audience and subject
# Never print the raw token
```

Check Trust: Use the claims your CI actually emits. Repo, branch and environment rules must match.

Check Permissions: id-token: write allows requesting a token. Cloud-side roles still decide what it can do.

Apply the cause specific fix

Aws Assume A Role

```
# GitHub Actions job fragment
permissions:
  contents: read
  id-token: write
steps:
- uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: ${{ vars.AWS_ROLE_ARN }}
    aws-region: ap-south-1
```

Before The Yaml: Create the IAM OIDC provider, restrict the actual token subject and audience, and grant least privilege.

Azure Federated Identity

```
# Same GitHub job permissions
steps:
- uses: azure/login@v2
  with:
    client-id: ${{ vars.AZURE_CLIENT_ID }}
    tenant-id: ${{ vars.AZURE_TENANT_ID }}
    subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
```

Before The Yaml: Create an Entra federated credential and scoped Azure RBAC. IDs are identifiers, not client secrets.

Fix The Confirmed Cause

```
# GCP: GitHub Actions job fragment
permissions:
  contents: read
  id-token: write
steps:
- uses: actions/checkout@v4
- uses: google-github-actions/auth@v2
  with:
    workload_identity_provider: ${{ vars.GCP_WIF_PROVIDER }}
    service_account: ${{ vars.GCP_SERVICE_ACCOUNT }}
```

Gcp Prerequisite: Create the workload identity pool/provider, scoped claim conditions and service-account impersonation binding.

Common Principle: Trust the intended CI identity, then issue short-lived credentials. No stored cloud password is required for this flow.

Validate the result

```
# Verify identity without exposing tokens
aws sts get-caller-identity
az account show
# GCP: validate access to one allowed resource
```

Positive Test: A trusted deployment can perform its intended operation.

Negative Test: An untrusted repository or branch cannot obtain equivalent access.

Practice exercise

Create a lab identity that can read one non-sensitive resource. Prove a trusted job succeeds, and a deliberately untrusted branch or subject fails. Verify expiry without logging tokens.

Implementation notes

All examples are job fragments for GitHub Actions, not Azure Pipelines. Use just the relevant provider steps. Pin actions to reviewed immutable commits in production; tags here identify action families. Provide variables, cloud trust and SDK/CLI setup. Actual OIDC sub formats can vary with GitHub repository settings and creation date; match observed documented claims, not copied legacy strings. Read-only identity commands do not prove deployment authorization. Review malicious workflow and PR trust boundaries.

Interview question

Does id-token write permission grant access to the cloud? No. It permits a token request; cloud trust and authorization are still required.

Record what changed

- The symptom and affected user operation.
- Evidence supporting the root cause and alternatives ruled out.
- The changed configuration and the version or identity involved.
- Positive and negative tests, plus any remaining risks.

Example status

Examples are educational templates or fragments. They were not deployed against a live cloud account or production cluster. A successful syntax check is not proof of runtime compatibility. Test adapted configurations in an isolated environment first.

Official references

Official documentation 1

Official documentation 2

Official documentation 3